

Практическая работа №2

Работа с командной оболочкой `bash` в ОС семейства Linux

Введение

Командная оболочка (command shell) предназначена для того, чтобы обеспечивать взаимодействие пользователя и ОС через командную строку. В разных дистрибутивах ОС семейства Linux могут применяться разные командные оболочки. Видов командных оболочек, конечно, несравнимо меньше, чем количество видов дистрибутивов Linux, однако с большой долей вероятности на самых популярных дистрибутивах вам попадется оболочка `bash`.

В отличие от графического интерфейса, командная оболочка – это универсальный способ взаимодействия с Linux-системами. Более того, командная оболочка дает самые широкие возможности по настройке, управлению ОС, а также выполнению рутинных задач.

Доступ к командной оболочке осуществляется через виртуальные терминалы либо через эмулятор терминала. В данной работе мы будем работать с эмулятором терминала и для краткости в дальнейшем будем называть его просто «терминал».

Терминал работает по принципу REPL (Read-Eval-Print-Loop). Фактически взаимодействие с пользователем осуществляется по следующему сценарию:

- 1) Read – оболочка ждёт ввода команды от пользователя.
- 2) Eval – оболочка исполняет введенную команду.
- 3) Print – оболочка выводит результат.
- 4) Loop – возвращаемся к первому пункту.

Полезные ссылки:

- Поучительная притча <https://www.livelib.ru/quote/61563-iskusstvo-programmirovaniya-dlya-unix-erik-c-rejmond>.
- Для тех, кто хочет глубже разобраться с терминалами <https://habr.com/ru/company/neobit/blog/330764/>.
- Удобный сайт, объясняющий введенную команду <https://explainshell.com/>.

Важно! Далее по тексту в строках, где указываются команды, которые необходимо вводить в терминал, знак \$ вводить не нужно. Этим знаком обозначается обычное приглашение консоли.

Описание практической работы

Цель работы

Научиться работать с командной оболочкой `bash` операционной системы семейства Linux и изучить основные команды файловой системы.

Указания к выполнению работы

Работа состоит из четырех частей. В первой части выполняется запуск и подготовка виртуального терминала, во второй части выполняются команды навигации по файловой системе, в третьей части изучаются команды манипулирования файлами, в четвертой части исследуются ключи и аргументы команд.

Оформление отчета

Отчёт должен быть составлен в соответствии с требованиями, представленными во введении. Экспериментальную часть работы необходимо сопроводить несколькими снимками экрана. Весь процесс фиксировать снимками экрана не нужно. В отчете необходимо дать ответы на вопросы, которые задаются по ходу работы, а также добавить требуемые данные (например, содержимое секретного файла). К отчету следует приложить файл `record_result`. Это текстовый файл с последовательностью выполненных команд.

Ход работы

Подготовка

- 1) Запустите виртуальную машину и дождитесь загрузки ОС.
- 2) Откройте терминал. Это можно сделать с помощью основного меню или сочетанием клавиш **Ctrl+Alt+T**. Для запуска через основное меню нажмите на Menu → System Tools → Mate Terminal (см. рисунок 2.1)

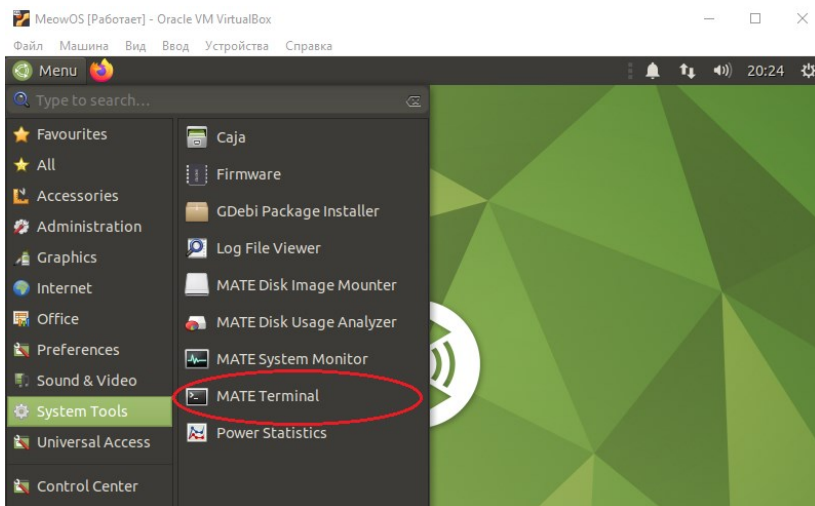


Рисунок 2.1 – Открытие терминала через графический интерфейс

- 3) **Запустите запись терминала. Это важно!** Именно файл с записью истории ваших команд будет основной частью отчета. Для этого введите и запустите команду

```
$ script -a record_result
```

- 4) Убедитесь, что команда выполнилась без ошибок. Введите нижеуказанную команду и найдите имя файла `record_result` в ее выводе:

```
$ ls -l
```

Навигация

- 1) Поместите на рабочий стол файл `lab2.py` (см. приложение А).
- 2) Вернитесь в терминал, в котором уже идет запись (см. Подготовка).
- 3) Выведите путь текущей рабочей директории. Для этого введите:

```
$ pwd
```

- 4) Выведите список всех файлов и каталогов в этой директории. Для этого введите:

```
$ ls -l
```

- 5) Перейдите на рабочий стол. Для этого введите:

```
$ cd ./Desktop
```

в случае с русским языком нужно использовать экранирование пробела:

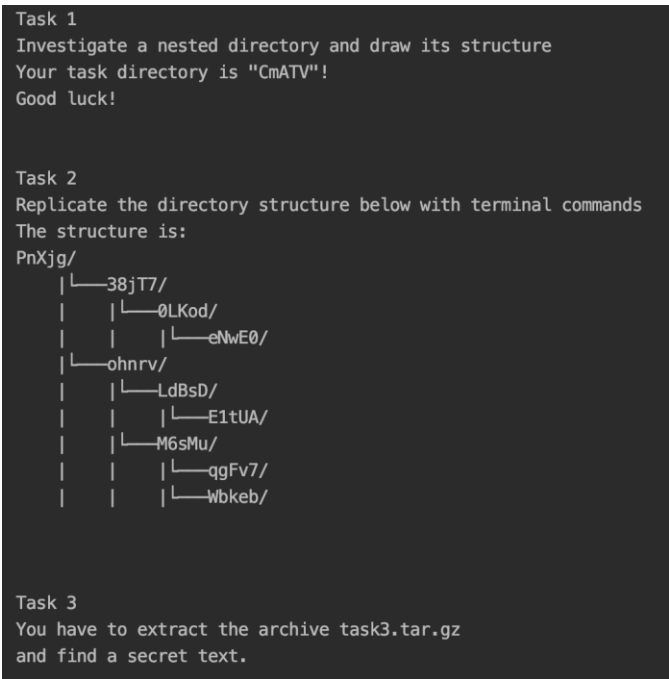
```
$ cd ./Рабочий\ стол/
```

- 6) Выведите список всех файлов и каталогов. Убедитесь, что в выводе есть файл `lab2.py`.

```
$ ls -l
```

- 7) Запустите генерацию вашего варианта (см. рисунок 2.2). Вывод этой команды добавьте в отчет в виде скриншота. Для генерации:

```
$ python lab2.py
```



```
Task 1
Investigate a nested directory and draw its structure
Your task directory is "CmATV"!
Good luck!

Task 2
Replicate the directory structure below with terminal commands
The structure is:
PnXjg/
|  └─38jT7/
|    └─0LKod/
|      └─eNwE0/
|  └─ohnrv/
|    └─LdBsD/
|      └─E1tUA/
|        └─M6sMu/
|          └─qgFv7/
|            └─Wbkeb/

Task 3
You have to extract the archive task3.tar.gz
and find a secret text.
```

Рисунок 2.2 – Результат генерации варианта

- 8) Посмотрите на задание Task 1. Там указано название директории, которую вам надо исследовать. Task 2 и Task 3 нам понадобятся позднее, пока они не нужны.
- 9) Исследуйте структуру созданной директории через терминал. Для этого используйте следующие команды:

```
$ ls -l # перечисляет папки и файлы рабочей директории
$ cd my_dir # переходит в директорию my_dir
$ cd /absolute/path # можно перейти по абсолютному пути
$ cd ./relative/path # можно перейти по относительному пути
$ cd .. # вернуться на уровень вверх
$ cd ../../.. # вернуться на два уровня вверх
$ cd ~ # перейти в домашнюю директорию
```

- 10) Среди директорий найдите текстовый файл. Выведите его содержимое в терминал. Его содержимое добавьте в отчет. Для вывода:

```
$ cat имя_найденного_файла
```

- 11) Нарисуйте дерево, соответствующее структуре исследованной вами папки, любым удобным способом и поместите в отчет.

Манипулирование файлами

- 1) Посмотрите на раздел Task 2 вашего варианта, который вывела нам ранее запущенная команда генерации заданий `python lab2.py`.
- 2) Перед собой вы увидите дерево директорий, представленное в символьном виде в терминале (см. рисунок 2.2).
- 3) Создайте соответствующую дереву структуру директорий с помощью терминала. Для этого вам понадобятся команды:

```
$ mkdir dir_name # создает директорию
$ mkdir -p вложенные/папки/ # рекурсивно создает директорию
$ cd dir_name # перейти в директорию
$ ls -l # перечисляет папки и файлы рабочей директории
$ rm dir_name # удаляет директорию
```

```
$ rm -rf dir_name # удаляет директорию рекурсивно
```

- 4) Перейдите в корневую папку вашего дерева.
- 5) В корневой папке создайте три файла следующей командой:

```
$ touch фамилия_1 фамилия_2 фамилия_3
```

- 6) Откройте каждый файл и запишите в него что-нибудь. Для этого откройте консольный редактор nano. Внесите произвольные изменения. Чтобы сохранить, нажмите `ctrl+x`, затем «Y»:

```
$ nano file_name
```

- 7) Файл фамилия_1 скопируйте и сохраните в той же папке. Для этого введите:

```
$ cp фамилия_1 copy_name
```

- 8) Файл фамилия_2 переместите в папку на уровень ниже (глубже) и переименуйте. Для этого:

```
$ mv фамилия_2 ./dir_name/new_name_2
```

- 9) Удалите файл фамилия_3:

```
$ rm -rf фамилия_3
```

- 10) Удалите рекурсивно все файлы, название которых заканчивается на цифру

```
$ find . -type f -name '[0-9]' -delete
```

Ключи и аргументы команд

- 1) Посмотрите справку команды `tar`. Для этого воспользуйтесь следующими командами справки:

```
$ tar --help
```

```
$ man tar
```

```
$ info tar
```

- 2) Изучите формат команды. Ответьте на вопросы. Какой ключ используется, чтобы создать архив? Какой ключ используется, чтобы извлечь архив? Какой ключ используется для выбора формата компрессии? Какой ключ используется для установки имени архива/директории?

- 3) Создайте архив `my_new_archive.tar.gz` из директории, созданной в предыдущем разделе «Манипулирование файлами» с использованием команды `tar`. Прикрепите его к отчету.
- 4) Извлеките файл из архива, созданного в предыдущем разделе «Навигация», в `Task 3` с использованием команды `tar`. Прочитайте содержимое `secret_file` из архива и добавьте в отчет.
- 5) Завершите запись команд, набрав в терминале:

`$ exit`

Контрольные вопросы

- 1) Как расшифровывается аббревиатура `bash`?
- 2) Что такое виртуальный терминал?
- 3) В чем отличие системных и встроенных команд оболочки?
- 4) Что такое архивация, чем она отличается от сжатия?
- 5) Что такое текущая рабочая директория?

Исходный код программы для практической работы №2

```
import os
import pathlib
import random
import string
import shutil
import tarfile

NAME_LEN = 5
MAX_DEPTH = 3
MAX_WIDTH = 3
random_choice = [True, False]

def generate_name():
    name = random.choices(string.digits + string.ascii_letters,
                           k=NAME_LEN)
    name = ''.join(name)
    return name

def make_dir(name):
    path = pathlib.Path.cwd().joinpath(name)
    path.mkdir(parents=True, exist_ok=True)

def generate_nested(depth=1, max_width=MAX_WIDTH,
max_depth=MAX_DEPTH):
    start = 2 if depth == 1 else 1
    number = random.randrange(start, max_width)
    for i in range(number):
        name = generate_name()
        make_dir(name)
        os.chdir(f'./{name}')
        if depth >= max_depth:
            if random.choice(random_choice):
                secret = generate_name()
                with open(secret, 'w') as f:
                    f.write(secret)
                random_choice.remove(True)
            os.chdir('..')
        else:
            depth = generate_nested(depth + 1)
```



```

os.chdir('..')
return depth - 1

def generate_text_struct():
    print('Task 2')
    print('Replicate the directory structure below with terminal commands ')
    print('The structure is:')
    name = generate_name()
    make_dir(name)
    startpath = f'./{name}'
    os.chdir(startpath)
    generate_nested()
    for root, dirs, files in os.walk(startpath):
        level = root.replace(startpath, '').count(os.sep)
        indent = '    |' * (level) + '└──' if level != 0 else '    '
        print('{}{}/'.format(indent, os.path.basename(root)))
        subindent = '|──' * (level + 1)
        for f in files:
            print('{}{}/'.format(subindent, f))
    shutil.rmtree(startpath)
    print('\n\n')

def generate_dirs():
    name = generate_name()
    make_dir(name)
    os.chdir(f'./{name}')
    generate_nested()
    print('Task 1')
    print('Investigate a nested directory and draw its structure')
    print(f'Your task directory is "{name}"!')
    print('Good luck!\n\n')

def generate_archive():
    cite = ['Отвлекись от всего, Нео! ',
            'Следуй за белым кроликом.',
            'Матрица повсюду. Она окружает нас.',
            'Ты только раб, Нео. Как и все, ты с рождения в цепях.',
            'Ты веришь в судьбу, Нео?',
            'Ложки не существует.',
            'Продам гараж!']

```

```

with open('secret_file', 'w') as f:
    f.write(random.choice(cite))

archive_name = 'task3.tar.gz'
tar = tarfile.open(archive_name, 'w:gz')
for name in ['secret_file']:
    tar.add(name)
tar.close()

os.remove('secret_file')

print('Task 3')
print(f'You have to extract the archive {archive_name}')
print('and find a secret text.')

if __name__ == '__main__':
    generate_dirs()
    generate_text_struct()
    generate_archive()

```